

Memoria del Proyecto: Flash Downloader

Título: Flash Downloader

Nombre: Lois Figueira, Óscar Rial, David Márquez

Módulo y curso: PSP | 2ºDAM

Fecha de entrega: 18/01/2026

Índice

1. Introducción	4
2. Tecnologías usadas y arquitectura del sistema	4
2.1 Tecnologías usadas	4
2.2 Arquitectura del sistema	5
3. Funcionalidades	5
4. Manual de usuario	13
4.1 Abrir desde un IDE	13
4.2 Instalar en Windows	13
4.3 Instalar en Android	13
4.4 Manual de uso	14
4.4.1 Interfaz Principal	14
4.4.2 Añadir Descargas	14
4.4.3 Gestión y Control	15
4.4.4 Configuración	15
5. Pruebas	15
6. Conclusiones y dificultades encontradas	16
7. Referencias	17
7.1 Lenguaje y Frameworks Core	17
7.2 Herramientas de Desarrollo y Construcción	17
7.3 Librerías y Gestión de Datos	17
7.4 Infraestructura y Entorno de Ejecución	17
8. Anexos	18
8.1 Repositorio GitHub	18
8.2 Uso de la IA	18
Resumen de uso de IA	18
Ejemplos de interacción (Prompts y Resultados)	18
1. Gestión de Concurrencia y Bloqueos	18
2. Algoritmo de Limitación de Velocidad	19
3. Configuración Gradle Multiplatform	19
4. UI Reactiva y Temas	20
Conclusión del anexo	20

8.3 Reparto del trabajo	20
-------------------------	----

1. Introducción

El proyecto consiste en la creación de **FlashDownloader**, una aplicación desarrollada en **Kotlin Multiplatform** para gestionar y acelerar la descarga de archivos desde internet, actuando como un gestor de descargas avanzado y centralizado.

Su funcionalidad principal permite al usuario pausar, reanudar, priorizar y organizar sus descargas, además de verificar la integridad de los archivos mediante Hash, superando así las capacidades básicas de los navegadores web convencionales.

Esta aplicación estará disponible tanto para dispositivos móviles (**Android**) como para sistemas de escritorio (**Windows, Linux, macOS**) y ofrecerá una interfaz gráfica moderna, unificada y adaptable gracias al uso de la tecnología **Compose Multiplatform**.

2. Tecnologías usadas y arquitectura del sistema

2.1 Tecnologías usadas

- **Kotlin Multiplatform (KMP):** Lenguaje principal del proyecto. Permite compartir toda la lógica de negocio y la capa de datos entre la aplicación de escritorio y la de Android, evitando duplicidad de código.
- **IntelliJ IDEA:** Entorno de desarrollo utilizado para la creación de la aplicación.
- **Compose Multiplatform:** Framework de interfaz gráfica declarado por JetBrains. Se utiliza para construir una UI moderna y unificada que funciona tanto en Windows/Linux/macOS como en Android.
- **Ktor Client:** Cliente HTTP asíncrono. Es el núcleo del gestor de descargas, configurado con el motor **CIO** para gestionar múltiples conexiones simultáneas y descargas de gran tamaño de forma eficiente.
- **Koin:** Framework de inyección de dependencias. Facilita la gestión de componentes y la modularización de la aplicación, desacoplando la lógica de la creación de objetos.

- **GitHub:** Sistema de control de versiones utilizado donde se aloja el proyecto remotamente.
- **Kotlin Coroutines:** Librería fundamental para la programación asíncrona. Gestiona los hilos de descarga en segundo plano para asegurar que la interfaz de usuario nunca se congele.
- **Compose Wizard:** Herramienta para hacer el proyecto multiplataforma.
- **Gradle:** Herramienta de construcción del proyecto, utilizada para gestionar las dependencias y compilar la aplicación para las distintas plataformas.
- **Java (JDK 11):** Entorno de ejecución base para asegurar la compatibilidad y estabilidad en la versión de escritorio.

2.2 Arquitectura del sistema

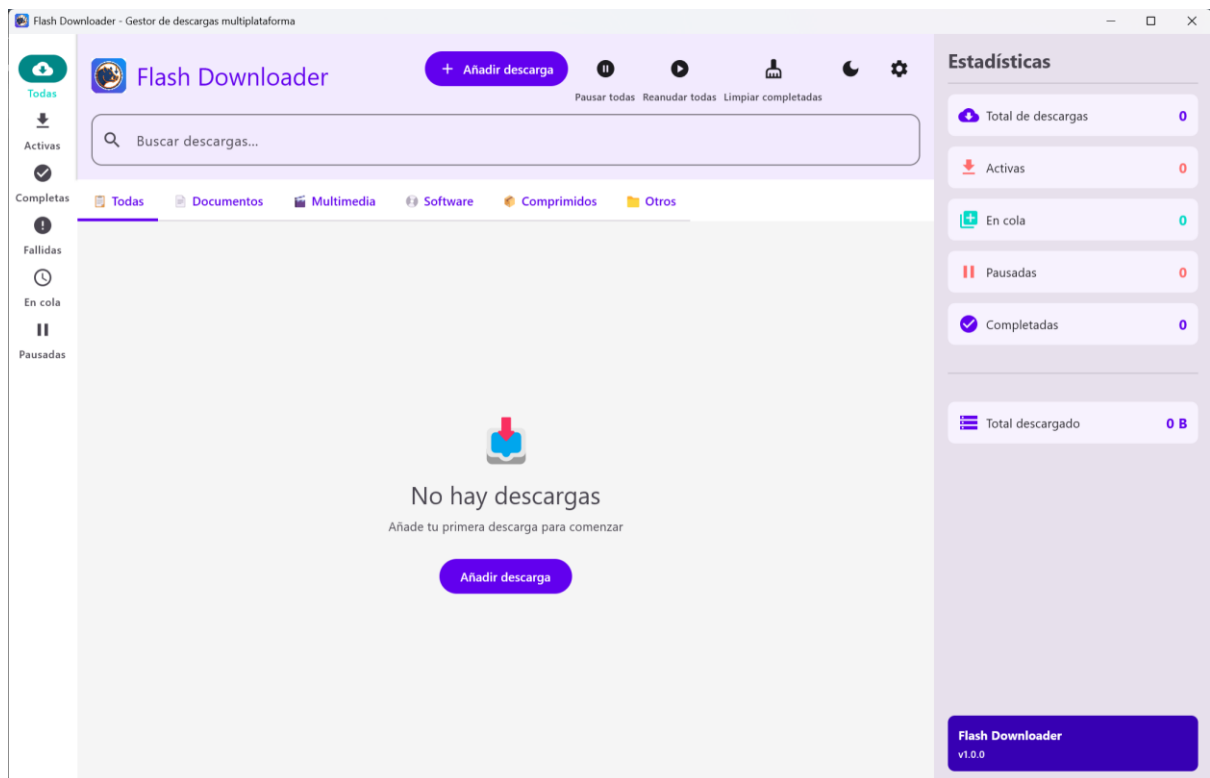
El proyecto implementa una **Clean Architecture** (Arquitectura Limpia) junto con el patrón **MVVM (Model-View-ViewModel)**. Esta estructura se divide en tres capas principales para facilitar el mantenimiento y la escalabilidad:

- **Capa de Presentación (UI):** Contiene las vistas en Compose y los ViewModels. Se encarga de mostrar los datos al usuario y capturar sus interacciones.
- **Capa de Dominio:** Define la lógica de negocio pura y las reglas de la aplicación (interfaces de repositorios y modelos), siendo independiente de cualquier librería externa o framework.
- **Capa de Datos:** Implementa la comunicación con el exterior (red y almacenamiento). Aquí reside la implementación técnica del cliente de descargas y el manejo de archivos.

3. Funcionalidades

- **Gestión de descargas:** Listar descargas mostrando nombre, progreso, velocidad y estado actual. Permite pausar, reanudar o cancelar descargas individualmente o en grupo.

- **Añadir recursos:** Captura de enlaces (manual o desde el portapapeles) con opciones para establecer prioridad, categoría y Hash de verificación.
- **Organización:** Filtrado de la lista por categorías (Música, Vídeo, Documentos, etc.) o por estado (En cola, Descargando, Completado).
- **Panel de control:** Estadísticas en tiempo real con la velocidad global de descarga y el número de descargas activas.
- **Integridad y Sistema:** Verificación de archivos corruptos mediante Hash (SHA-256) y persistencia de datos automática.



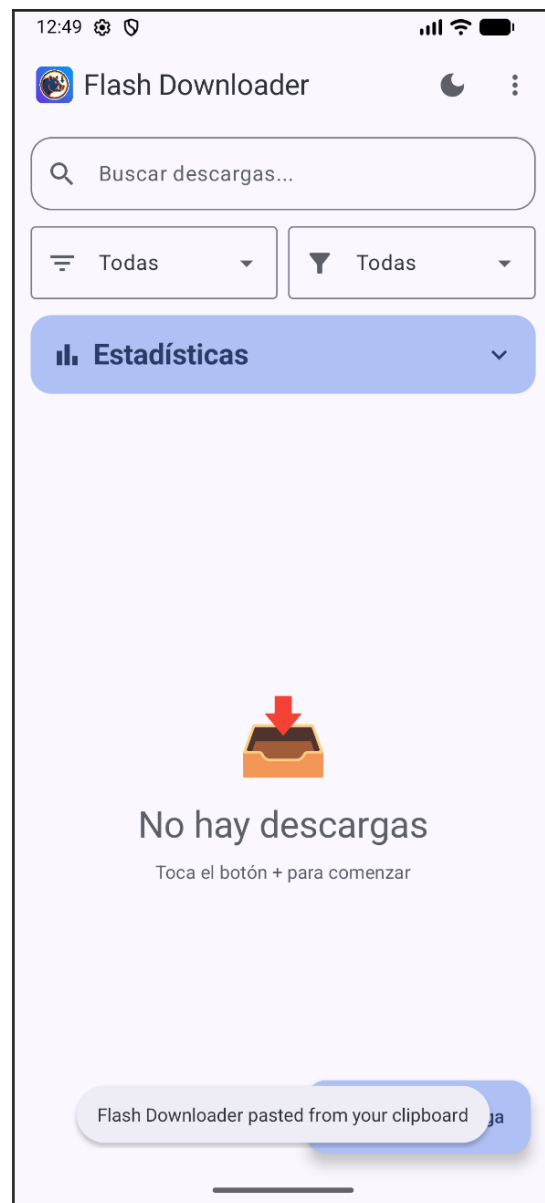
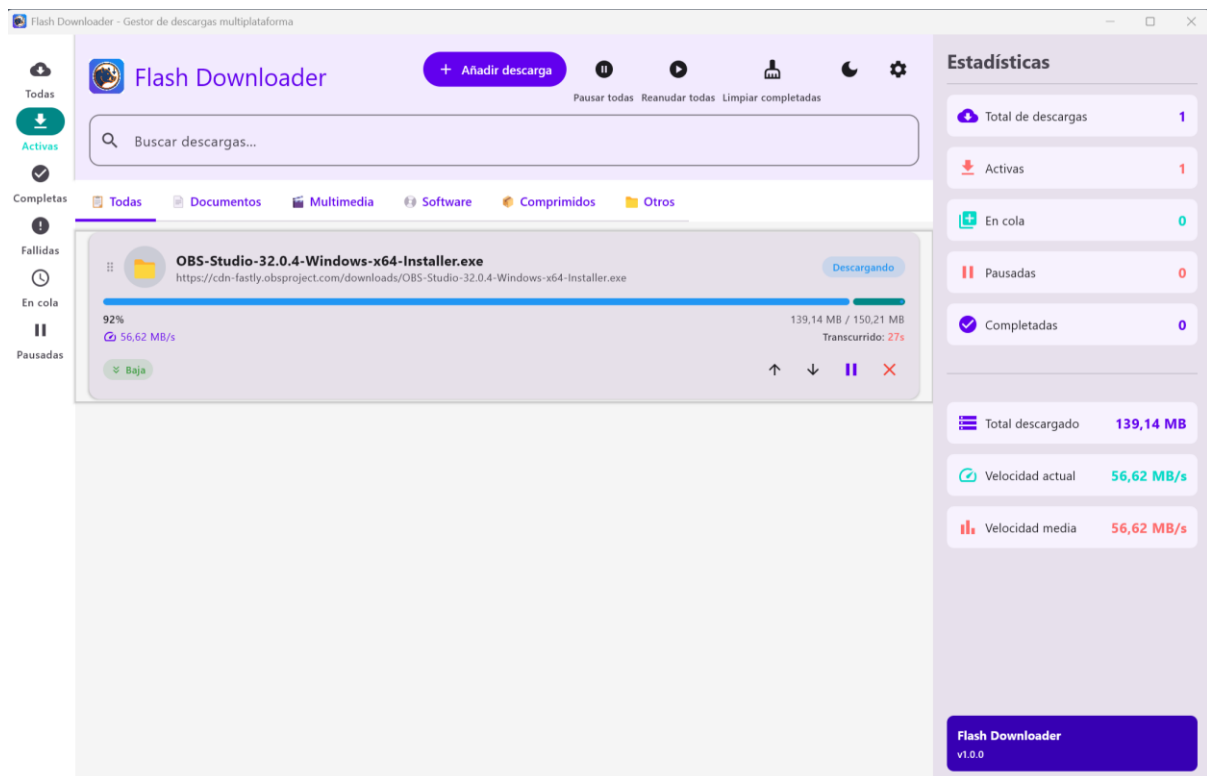


Imagen 1: Captura de pantalla de la interfaz de usuario

Descripción: Esta imagen muestra la pantalla principal donde se visualiza la lista de descargas activas con sus barras de progreso. En la parte superior (o lateral en escritorio) destaca el panel de estadísticas con la velocidad global, y en la barra de herramientas se observan las opciones para pausar o reanudar todas las tareas simultáneamente.



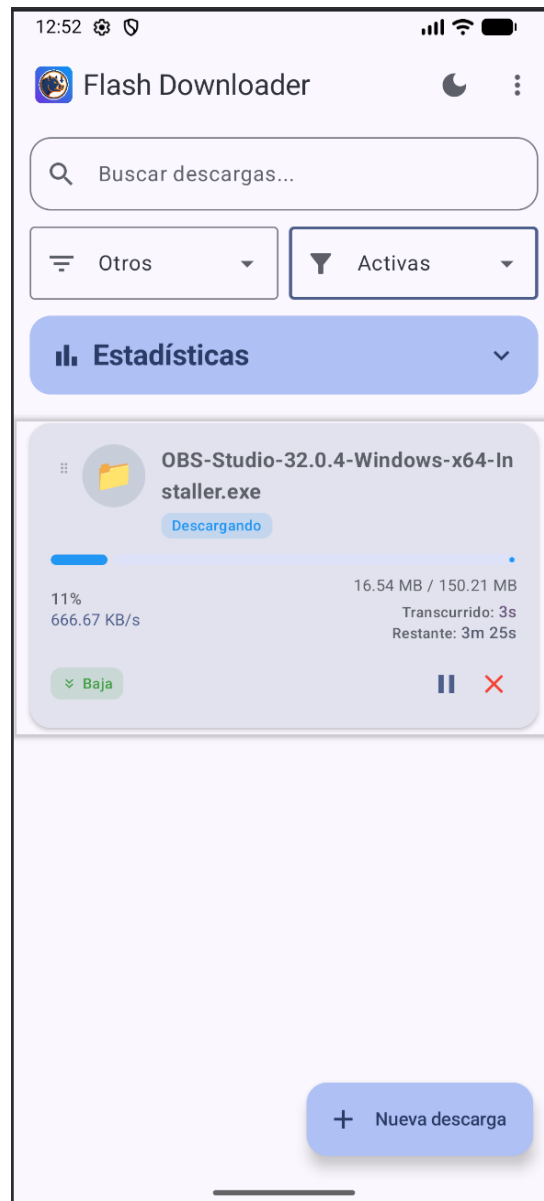


Imagen 2: Captura de pantalla del uso de los filtros

Descripción: En esta captura se muestra cómo se organiza la lista al aplicar filtros. Por ejemplo, al seleccionar la categoría "Comprimidos" o el estado "En cola", la interfaz oculta el resto de elementos para facilitar la búsqueda de un archivo específico.

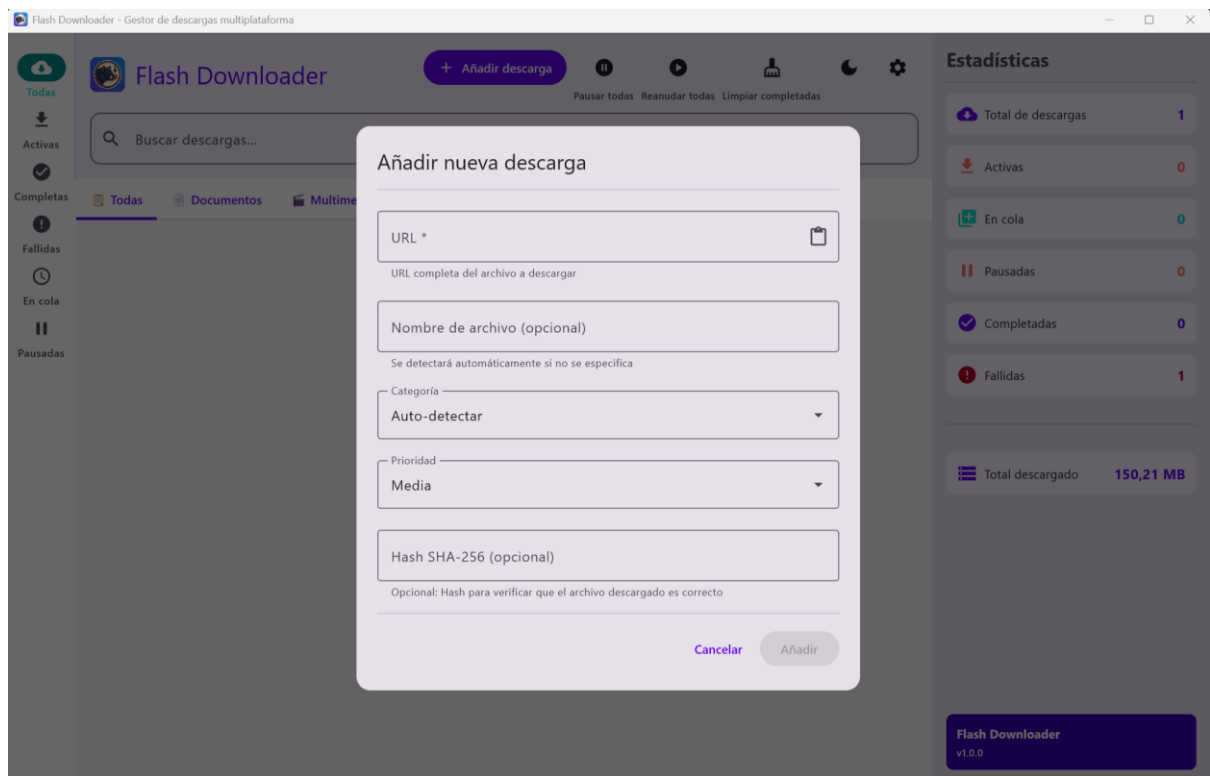




Imagen 3: Captura de pantalla del diálogo de añadir descarga

Descripción: En esta captura se muestra el formulario para iniciar una nueva descarga. Se observan los campos para pegar la URL, el selector de prioridad (Baja, Media, Alta) y el campo opcional para introducir el Hash de verificación, que asegura que el archivo no esté corrupto al finalizar.

4. Manual de usuario

4.1 Abrir desde un IDE

1. Para hacer uso del proyecto desde un IDE como IntelliJ IDEA, se deberá clonar el repositorio en el enlace proporcionado en el anexo. Desde un terminal se haría "git clone enlace_al_repo".
2. Una vez termina de clonar ya se puede usar en el botón de Play (ejecutar) dentro de un IDE.

4.2 Instalar en Windows

Instalación utilizando el ejecutable:

1. Se accede al repositorio y en el apartado releases se descarga el .msi.
2. Luego se sigue el proceso de instalación.
3. Por último, se busca una carpeta con el nombre "com.dam2.flashdownloader", donde se encuentra la aplicación.

4.3 Instalar en Android

1. Se accede al repositorio y en el apartado releases se descarga el .apk.
2. Una vez hecho se localiza (probablemente en Descargas) y se ejecuta el .apk, se instalará automáticamente.
3. Una vez hecho, la aplicación ya aparecerá entre las aplicaciones y se podrá utilizar.

4.4 Manual de uso

El Manual de usuario de FlashDownloader describe cómo interactuar con la interfaz unificada de la aplicación para gestionar descargas de forma eficiente.

4.4.1 Interfaz Principal

Al iniciar la aplicación, se presenta un panel de control dividido en tres secciones estratégicas para facilitar el flujo de trabajo:

- **Barra Lateral (Izquierda):** Permite filtrar la vista actual según el estado de las descargas (Activas, En Cola, Pausadas, Completadas, Fallidas) o por categoría de archivo.
- **Lista Central:** Muestra las descargas actuales. Cada tarjeta ofrece información visual inmediata sobre el progreso, velocidad actual y tamaño del archivo. También dispone de una barra de búsqueda superior para localizar archivos específicos por nombre o URL.
- **Panel de Estadísticas (Derecha):** Ofrece métricas en tiempo real de la sesión, incluyendo la velocidad global de descarga, el número de descargas activas y el volumen total de datos transferidos.

4.4.2 Añadir Descargas

Existen dos métodos para iniciar una descarga:

- **Detección Inteligente:** Si la opción está activada en ajustes, FlashDownloader detectará automáticamente si copias un enlace válido en el portapapeles y te sugerirá añadirlo.
- **Botón "Añadir Descarga":** Abre un formulario donde se pueden configurar los detalles:
 - **Enlace (URL):** Único campo obligatorio.
 - **Nombre y Categoría:** Se autocompletan automáticamente, pero son editables.
 - **Prioridad:** Permite establecer la importancia (Alta, Media, Baja) para ordenar la cola.
 - **Hash (SHA-256):** Campo opcional para verificar la integridad del archivo una vez descargado, asegurando que no está corrupto.

4.4.3 Gestión y Control

El usuario tiene control total sobre el flujo de descargas mediante acciones globales e individuales:

- **Controles Globales:** Botones de acceso rápido para "Pausar todo", "Reanudar todo" o "Limpiar completadas" para mantener la lista ordenada.
- **Acciones Individuales:** Cada descarga permite pausar, reanudar, cancelar o abrir la carpeta de destino una vez finalizada.
- **Integridad:** Si se proporcionó un Hash, el sistema indicará visualmente si la verificación fue exitosa o fallida.

4.4.4 Configuración

Desde el menú de ajustes (icono de engranaje) se puede personalizar el comportamiento de la aplicación:

- **Apariencia:** Alternar entre Tema Claro y Tema Oscuro.
- **Concurrencia:** Ajustar el número máximo de descargas simultáneas (1-10) para no saturar el equipo.
- **Límite de Velocidad:** Establecer un límite global de ancho de banda para evitar que la aplicación consuma toda la red.

5. Pruebas

Se han realizado pruebas funcionales y unitarias para verificar la estabilidad de la aplicación en los entornos objetivo.

- **Compatibilidad:** Se ha comprobado el correcto funcionamiento de la interfaz y el servicio de descargas tanto en Windows como en dispositivos Android, verificando la adaptabilidad de la UI.
- **Gestión de descargas:** Se verificó la capacidad de pausar, reanudar y cancelar descargas sin pérdida de datos, así como la gestión correcta de la cola de prioridades.
- **Rendimiento y Estrés:** Se realizaron pruebas con archivos de gran tamaño (+2GB) y con limitación de ancho de banda activada para asegurar que la aplicación no se congela ni consume memoria excesiva.

- **Integridad de datos:** Se comprobó que el sistema verifica correctamente la integridad de los archivos descargados mediante SHA-256, alertando si el archivo está corrupto.
- **Pruebas Unitarias:** Se ejecutaron test automatizados para validar la lógica interna crítica, como los modelos de datos y los algoritmos de limitación de velocidad.

6. Conclusiones y dificultades encontradas

Durante el desarrollo de FlashDownloader hemos aprendido mucho como equipo y nos hemos enfrentado a varios retos técnicos que tuvimos que resolver juntos.

En cuanto al aprendizaje:

- Hemos aprendido a crear una aplicación que funciona tanto en Android como en PC con el mismo código base, algo nuevo para nosotros.
- Descubrimos cómo usar Compose para diseñar una interfaz moderna que se adapta bien a diferentes pantallas.
- También aprendimos a controlar múltiples descargas simultáneas en segundo plano sin que la aplicación se congele.

En cuanto a dificultades:

- Lo más complicado para el grupo fue lograr que las descargas se pudieran pausar y reanudar correctamente sin romper los archivos. Nos costó coordinar esa lógica para que fuera estable.
- Tuvimos bastantes problemas configurando Gradle para que el proyecto compilara bien en todos nuestros entornos (Windows/Android) sin conflictos de versiones.
- Otro reto importante fue evitar que la aplicación se cerrara al descargar archivos muy grandes (+2GB); tuvimos que investigar y ajustar la memoria para solucionarlo.

En resumen, lo más laborioso fue la configuración inicial del entorno multiplataforma y conseguir que el sistema de descargas fuera robusto ante interrupciones.

7. Referencias

7.1 Lenguaje y Frameworks Core

- **Kotlin Multiplatform (KMP):** JetBrains. (2024). Kotlin Multiplatform Development Documentation - <https://kotlinlang.org/docs/multiplatform.html>
- **Compose Multiplatform:** JetBrains. (2024). Compose Multiplatform UI Framework - <https://www.jetbrains.com/lp/compose-multiplatform/>
- **Kotlin Coroutines:** Elizarov, R. (2017). Guide to UI programming with coroutines. JetBrains - <https://kotlinlang.org/docs/coroutines-overview.html>

7.2 Herramientas de Desarrollo y Construcción

- **IntelliJ IDEA:** JetBrains. (2024). IntelliJ IDEA: The Capable & Ergonomic IDE for JVM - <https://www.jetbrains.com/idea/>
- **Gradle:** Gradle Inc. (2024). Gradle Build Tool Documentation - <https://docs.gradle.org>
- **Compose Wizard:** JetBrains. (2024). Kotlin Multiplatform Wizard - <https://terrakok.github.io/Compose-Multiplatform-Wizard/>

7.3 Librerías y Gestión de Datos

- **Ktor Client:** JetBrains. (2024). Ktor: Build asynchronous servers and clients - <https://ktor.io/docs/client-engines.html>
- **Koin Dependency Injection:** Kotzilla. (2024). Koin Documentation: The pragmatic Kotlin Dependency Injection framework - <https://insert-koin.io/>

7.4 Infraestructura y Entorno de Ejecución

- **Java Development Kit (JDK 11):** Oracle Corp. (2018). Java SE 11 Archive Downloads - <https://www.oracle.com/java/technologies/javase-jdk11-downloads.html>
- **GitHub:** GitHub, Inc. (2024). Enlace al repositorio - <https://github.com/oscarclase23/FlashDownloader/tree/pro>

8. Anexos

8.1 Repositorio GitHub

Enlace al repositorio de GitHub donde se aloja el proyecto y los instaladores.

<https://github.com/oscarclase23/FlashDownloader/tree/pro>

8.2 Uso de la IA

Resumen de uso de IA

Para el desarrollo de **FlashDownloader**, el equipo se ha apoyado en un ecosistema de herramientas de Inteligencia Artificial compuesto por **ChatGPT**, **Claude**, **Gemini** y el asistente integrado en el IDE, **Antigravity**.

Estas herramientas han actuado como "programadores de apoyo" (pair programming), acelerando significativamente el flujo de trabajo en áreas críticas:

- **Claude:** Se utilizó principalmente para la arquitectura del sistema y refactorización de código complejo. Fue clave para diseñar la lógica de la **Clean Architecture** y la separación de capas (Domain, Data, Presentation).
- **ChatGPT y Gemini:** Se emplearon para la resolución rápida de errores de compilación en **Gradle**, la generación de componentes visuales en **Compose Multiplatform** y la consulta de documentación sobre librerías específicas como Ktor y Koin.
- **Antigravity (IDE):** Fue fundamental para la escritura de código repetitivo (boilerplate) y la autocompletación inteligente en tiempo real, especialmente al escribir los tests unitarios y las implementaciones de interfaces.

Ejemplos de interacción (Prompts y Resultados)

A continuación, se detallan ejemplos concretos de cómo la IA ayudó a resolver los desafíos más técnicos del proyecto:

1. Gestión de Concurrencia y Bloqueos

Contexto: Necesitábamos gestionar múltiples descargas simultáneas sin que la modificación de la lista provocara errores de acceso concurrente (*race conditions*).

- **Prompt (Claude):**

"Tengo una lista mutable de descargas en un `DownloadManager` que se accede desde varios hilos. Necesito asegurar que sea thread-safe pero usando Coroutines,

y también necesito limitar el número de descargas activas a 3. ¿Cómo puedo implementar esto usando Mutex y Semaphore en lugar de bloqueos tradicionales de Java?"

- **Respuesta y Solución aplicada:** La IA sugirió proteger la lista `_downloadsList` con un Mutex para garantizar la exclusión mutua durante las operaciones de lectura/escritura y utilizar un Semaphore para controlar el número de descargas activas. Esto se implementó directamente en la clase `DownloadManagerImpl` para evitar *deadlocks*.

2. Algoritmo de Limitación de Velocidad

Contexto: Requeríamos una forma de limitar la velocidad de descarga (bytes por segundo) de manera precisa.

- **Prompt (ChatGPT):**

"¿Cómo puedo implementar un limitador de ancho de banda en Kotlin para un flujo de descarga? Explícame el algoritmo 'Token Bucket' y dame un ejemplo de implementación que pueda pausar la corrutina si se excede el límite."

- **Respuesta y Solución aplicada:** La IA generó la clase base para el `TokenBucketLimiter`, explicando cómo "rellenar" los tokens periódicamente y suspender la ejecución con `delay()` si no hay suficientes tokens disponibles. Este código es la base de nuestra clase `BandwidthLimiter`.

3. Configuración Gradle Multiplatform

Contexto: Configurar las dependencias para que funcionen tanto en Android como en Desktop (JVM) suele dar muchos errores de resolución.

- **Prompt (Gemini):**

"Estoy migrando a Version Catalogs (`libs.versions.toml`) en un proyecto Kotlin Multiplatform. Tengo problemas con la dependencia de `compose.desktop.currentOs` y `ktor-client-cio` en el bloque `jvmMain`. ¿Cómo debo declarar estas librerías correctamente en el `build.gradle.kts`?"

- **Respuesta y Solución aplicada:** Gemini identificó que algunas dependencias de Compose deben declararse específicamente en el bloque de dependencias de escritorio y ayudó a estructurar correctamente el archivo `libs.versions.toml` para unificar versiones.

4. UI Reactiva y Temas

Contexto: Necesitábamos que la interfaz reaccionara a los cambios de estado y soportara modo oscuro.

- **Prompt (Antigravity/IDE):**

Escrito como comentario en el código: // Crear una StateFlow en el ViewModel que combine la lista de descargas con los filtros de búsqueda y categoría seleccionados.

- **Respuesta y Solución aplicada:** La IA del IDE autocompletó la lógica del combine en DownloadViewModel, generando el flujo filteredDownloads que actualiza la UI automáticamente cuando cambia el texto de búsqueda o el filtro de estado.

Conclusión del anexo

El uso combinado de **Claude** para arquitectura profunda, **Gemini/ChatGPT** para consultas puntuales y **Antigravity** para la escritura de código ha resultado en un desarrollo mucho más ágil y robusto. La IA no solo facilitó la resolución de problemas técnicos complejos (como la concurrencia o la configuración de Gradle), sino que actuó como una herramienta de aprendizaje continuo para el equipo, permitiéndonos implementar patrones avanzados de Kotlin que desconocíamos al inicio del proyecto.

8.3 Reparto del trabajo

Apartado del trabajo	Integrantes responsables
Desarrollo: Interfaz de Usuario (UI) (Implementación de DownloadApp, DownloadListItem, Diálogos de añadir/configuración, TopBar y Temas en Compose Multiplatform)	Todos (Oscar, Lois, David)
Desarrollo: Lógica de Negocio y Dominio (Implementación de DownloadManager, DownloadRepository, BandwidthLimiter y Modelos de datos)	Todos (Oscar, Lois, David)

Desarrollo: Implementación Específica de Plataforma (Configuración de AndroidMain (Services, Notifications) y JvmMain (FileWriters, System Tray))	Todos (Oscar, Lois, David)
Desarrollo: Utilidades y Algoritmos (Cálculo de Hashes, gestión de portapapeles y apertura de archivos)	Todos (Oscar, Lois, David)
Elaboración de Documentación (Redacción de memoria técnica, manuales y comentarios de código)	Oscar, Lois
Gestión de Releases y CI/CD (Configuración de GitHub Workflows, release.yml y empaquetado de versiones)	Todos (Oscar, Lois, David)
Presentación y Demostración (Grabación y edición del vídeo demostrativo del funcionamiento)	David